Distinguishing Access and Filter-Predicates

The PostgreSQL database uses three different methods to apply **where** clauses (predicates):

Access Predicate ("Index Cond")
: The access predicates express the start and stop conditions of the leaf node traversal.

Index Filter Predicate ("Index Cond")
: Index filter predicates are applied during the leaf node traversal only. They do not contribute to the start and stop conditions and do not narrow the scanned range.

Table level filter predicate ("Filter")
: Predicates on columns that are not part of the index are evaluated on the table level. For that to happen, the database must load the row from the heap table first.

PostgreSQL execution plans do not show index access and filter predicates separately—both show up as "Index Cond". That means the execution plan must be compared to the index definition to differentiate access predicates from index filter predicates.



Note

The PostgreSQL explain plan does not provide enough information for finding index filter predicates.

The predicates shown as "Filter" are always table level filter predicates—even when shown for an `Index Scan` operation.

Consider the following example, which originally appeared in the "Performance and Scalability" chapter:

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);
CREATE INDEX scale_data_key ON scale_data(section, id1);
```

The following **select** filters on the ID2 column, which is not included in the index:

```
PREPARE stmt(int) AS SELECT count(*)
                       FROM scale_data
                      WHERE section = 1
                        AND id2 = $1;
EXPLAIN EXECUTE stmt(1);

                      QUERY PLAN
------------------------------------------------------
Aggregate  (cost=529346.31..529346.32 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_data_key on scale_data
     (cost=0.00..529338.83 rows=2989 width=0)
     Index Cond: (scale_data.section = 1::numeric)
     Filter: (scale_data.id2 = ($1)::numeric)
```

The ID2 predicate shows up as "Filter" below the Index Scan operation. This is because PostgreSQL performs the table access as part of the Index Scan operation. In other words, the TABLE ACCESS BY INDEX ROWID operation of the Oracle database is hidden within PostgreSQL's Index Scan operation. It is therefore possible that a Index Scan filters on columns that are not included in the index.



Important

The PostgreSQL Filter predicates are table level filter predicates—even when shown for an Index Scan.

When we add the index from the "Performance and Scalability" chapter, we can see that all columns show up as "Index Cond"—regardless of whether they are access or filter predicates.

```
CREATE INDEX scale_slow
          ON scale_data (section, id1, id2);
```

The execution plan with the new index does not show any filter conditions:

```
                      QUERY PLAN
------------------------------------------------------
Aggregate  (cost=14215.98..14215.99 rows=1 width=0)
  Output: count(*)
  -> Index Scan using scale_slow on scale_data
     (cost=0.00..14208.51 rows=2989 width=0)
     Index Cond: (section = 1::numeric AND id2 = ($1)::numeric)
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Please note that the condition on `ID2` cannot narrow the leaf node traversal because the index has the `ID1` column before `ID2`. That means, the `Index Scan` will scan the entire range for the condition `SECTION=1::numeric` and apply the filter `ID2=($1)::numeric` on each row that fulfills the clause on `SECTION`.

SQL Server

The method described in this section applies to SQL Server Management Studio 2005 and later.

Getting an Execution Plan

With SQL Server, there are several ways to fetch an execution plan. The two most important methods are:

Graphically
: The graphical representation of SQL Server execution plans is easily accessible in the Management Studio but is hard to share because the predicate information is only visible when the mouse is moved over the particular operation ("hover").

Tabular
: The tabular execution plan is hard to read but easy to copy because it shows all relevant information at once.

Graphically

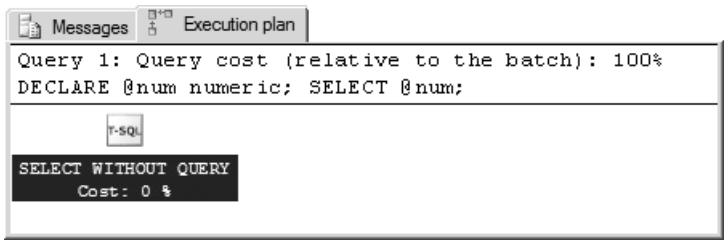
The graphical explain plan is generated with one of the two buttons highlighted below.



The left button explains the highlighted statement directly. The right will capture the plan the next time a SQL statement is executed.

In both cases, the graphical representation of the execution plan appears in the "Execution plan" tab of the "Results" pane.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>



The graphical representation is easy to read with a little bit of practice. Nonetheless, it only shows the most fundamental information: the operations and the table or index they act upon.

The Management Studio shows more information when moving the mouse over an operation (mouseover/hover). This makes it hard to share an execution plan with all its details.

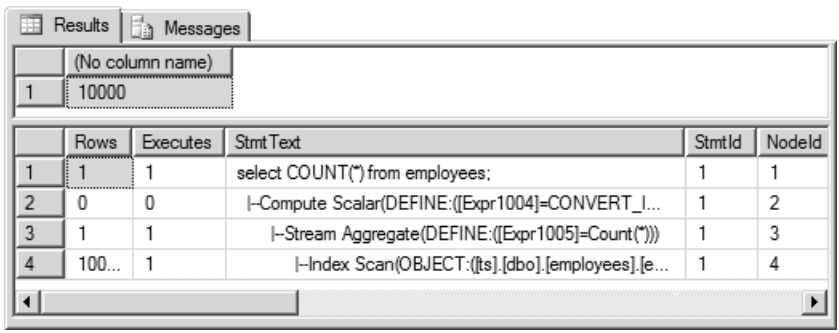
Tabular

The tabular representation of an SQL Server execution plan is fetched by profiling the execution of a statement. The following command enables it:

```
SET STATISTICS PROFILE ON
```

Once enabled, each executed statement produces an extra result set. **`select`** statements, for example, produce two result sets—the result of the statement first then the execution plan.

The tabular execution plan is hardly usable in SQL Server Management Studio because the `StmtText` is just too wide to fit on a screen.



The advantage of this representation is that it can be copied without loosing relevant information. This is very handy if you want to post an SQL Server execution plan on a forum or similar platform. In this case, it is often enough to copy the `StmtText` column and reformat it a little bit:

```
select COUNT(*) from employees;
  |--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
       |--Stream Aggregate(DEFINE:([Expr1005]=Count(*)))
            |--Index Scan(OBJECT:([employees].[employees_pk]))
```

Finally, you can disable the profiling again:

```
SET STATISTICS PROFILE OFF
```

Operations

Index and Table Access

SQL Server has a simple terminology: "Scan" operations read the entire index or table while "Seek" operations use the B-tree or a physical address (`RID`, like Oracle `ROWID`) to access a specific part of the index or table.

Index Seek

The `Index Seek` performs a B-tree traversal *and* walks through the leaf nodes to find all matching entries. See also "*Anatomy of an Index*" on page 1.

Index Scan

Reads the entire index—all the rows—in the index order. Depending on various system statistics, the database might perform this operation if it needs all rows in index order—e.g., because of a corresponding **`order by`** clause.

Key Lookup (Clustered)

Retrieves a single row from a clustered index. This is similar to Oracle `INDEX UNIQUE SCAN` for an Index-Organized-Table (IOT). See also "*Clustering Data*" on page 111.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

RID Lookup (Heap)
: Retrieves a single row from a table—like Oracle `TABLE ACCESS BY INDEX ROWID`. See also "*Anatomy of an Index*" on page 1.

Table Scan
: This is also known as full table scan. Reads the entire table—all rows and columns—as stored on the disk. Although multi-block read operations can improve the speed of a `Table Scan` considerably, it is still one of the most expensive operations. Besides high IO rates, a `Table Scan` must also inspect all table rows so it can also consume a considerable amount of CPU time. See also "Full Table Scan" on page 13.

Join Operations

Generally join operations process only two tables at a time. In case a query has more joins, they are executed sequentially: first two tables, then the intermediate result with the next table. In the context of joins, the term "table" could therefore also mean "intermediate result".

Nested Loops
: Joins two tables by fetching the result from one table and querying the other table for each row from the first. SQL Server also uses the nested loops operation to retrieve table data after an index access. See also "Nested Loops" on page 92.

Hash Match
: The hash match join loads the candidate records from one side of the join into a hash table which is then probed for each row from the other side of the join. See also "Hash Join" on page 101.

Merge Join
: The merge join combines two sorted lists like a zipper. Both sides of the join must be presorted. See also "Sort Merge" on page 109.

Sorting and Grouping

Sort
: Sorts the result according to the **`order by`** clause. This operation needs large amounts of memory to materialize the intermediate result (not pipelined). See also "Indexing Order By" on page 130.

Sort (Top N Sort)
: Sorts a subset of the result according to the **`order by`** clause. Used for top-N queries if pipelined execution is not possible. See also "Querying Top-N Rows" on page 143.

Stream Aggregate
: Aggregates a presorted set according the **`group by`** clause. This operation does not buffer the intermediate result—it is executed in a pipelined manner. See also "Indexing Group By" on page 139.

Hash Match (Aggregate)
: Groups the result using a hash table. This operation needs large amounts of memory to materialize the intermediate result (not pipelined). The output is not ordered in any meaningful way. See also "Indexing Group By" on page 139.

Top-N Queries

Top
: Aborts the underlying operations when the desired number of rows has been fetched. See also "Querying Top-N Rows" on page 143.

  The efficiency of the top-N query depends on the execution mode of the underlying operations. It is very inefficient when aborting non-pipelined operations such as `Sort`.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Distinguishing Access and Filter-Predicates

The SQL Server database uses three different methods for applying **where** clauses (predicates):

Access Predicate ("Seek Predicates")
: The access predicates express the start and stop conditions of the leaf node traversal.

Index Filter Predicate ("Predicates" or "where" for index operations)
: Index filter predicates are applied during the leaf node traversal only. They do not contribute to the start and stop conditions and do not narrow the scanned range.

Table level filter predicate ("where" for table operations)
: Predicates on columns which are not part of the index are evaluated on the table level. For that to happen, the database must load the row from the heap table first.

The following section explains how to identify filter predicates in SQL Server execution plans. It is based on the sample used to demonstrate the impact of index filter predicates in Chapter 3.

```
CREATE TABLE scale_data (
   section NUMERIC NOT NULL,
   id1     NUMERIC NOT NULL,
   id2     NUMERIC NOT NULL
);

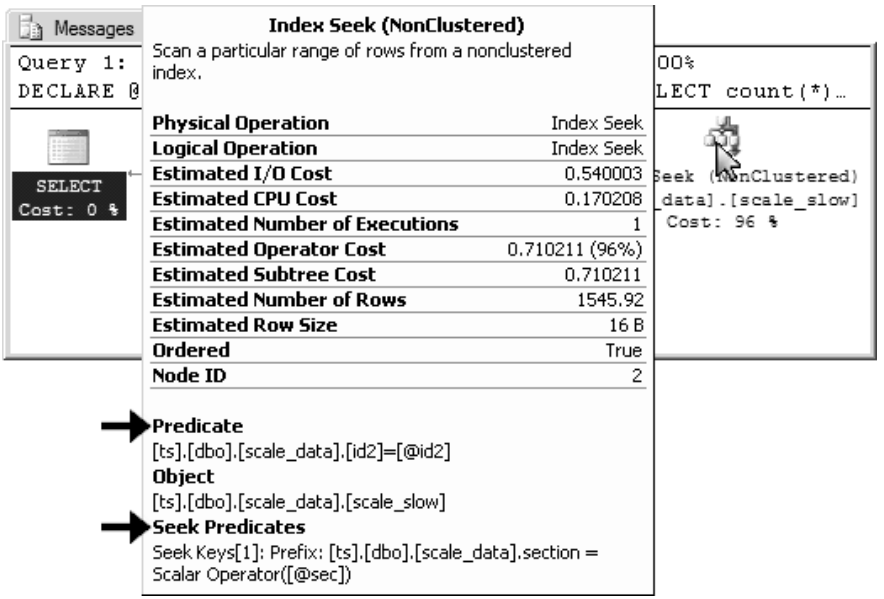
CREATE INDEX scale_slow ON scale_data(section, id1, id2);
```

The sample statement selects by `SECTION` and `ID2`:

```
SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2
```

IN GRAPHICAL EXECUTION PLANS

The graphical execution plan hides the predicate information in a tooltip that is only shown when moving the mouse over the `Index Seek` operation.



The SQL Server's *Seek Predicates* correspond to Oracle's access predicates—they narrow the leaf node traversal. Filter predicates are just labeled *Predicates* in SQL Server's graphical execution plan.

In Tabular Execution Plans

Tabular execution plans have the predicate information in the same column in which the operations appear. It is therefore very easy to copy and past all the relevant information in one go.

```
DECLARE @sec numeric;
DECLARE @id2 numeric;

SET STATISTICS PROFILE ON

SELECT count(*)
  FROM scale_data
 WHERE section = @sec
   AND id2 = @id2

SET STATISTICS PROFILE OFF
```

The execution plan is shown as a second result set in the results pane. The following is the `StmtText` column—with a little reformatting for better reading:

```
|--Compute Scalar(DEFINE:([Expr1004]=CONVERT_IMPLICIT(...))
     |--Stream Aggregate(DEFINE:([Expr1008]=Count(*)))
          |--Index Seek(OBJECT:([scale_data].[scale_slow]),
             SEEK: ([scale_data].[section]=[@sec])
                   ORDERED FORWARD
             WHERE:([scale_data].[id2]=[@id2]))
```

The `SEEK` label introduces access predicates, the `WHERE` label marks filter predicates.

MySQL

The method described in this section applies to all versions of MySQL.

Getting an Execution Plan

Put **explain** in front of an SQL statement to retrieve the execution plan.

```
EXPLAIN SELECT 1;
```

The plan is shown in tabular form (some less important columns removed):

```
~+-------+------+---------------+------+~+------+------------~
~| table | type | possible_keys | key  |~| rows | Extra
~+-------+------+---------------+------+~+------+------------~
~| NULL  | NULL | NULL          | NULL |~| NULL | No tables...
~+-------+------+---------------+------+~+------+------------~
```

The most important information is in the TYPE column. Although the MySQL documentation refers to it as "join type", I prefer to describe it as "access type" because it actually specifies how the data is accessed. The meaning of the type value is described in the next section.

Operations

Index and Table Access

MySQL's explain plan tends to give a false sense of safety because it says so much about indexes being used. Although technically correct, it does not mean that it is using the index efficiently. The most important information is in the TYPE column of the MySQL's **explain** output—but even there, the keyword INDEX doesn't indicate proper indexing.

eq_ref
: Performs a B-tree traversal only. The database uses this operation if a primary key or unique constraint ensures that the search criteria will match no more than one entry. See also "*Anatomy of an Index*" on page 1.

ref, range
: Performs a B-tree traversal *and* walks through the leaf nodes to find all matching entries (similar to `INDEX RANGE SCAN`). See also "*Anatomy of an Index*" on page 1.

index
: Reads the entire index—all rows—in the index order (similar to `INDEX FULL SCAN`).

ALL
: Reads the entire table—all rows and columns—as stored on the disk. Besides high IO rates, a table scan must also inspect all rows from the table so that it can also put a considerable load on the CPU. See also "Full Table Scan" on page 13.

Using Index (in the "Extra" column)
: When the "Extra" column shows "Using Index", it means that the table is not accessed because the index has all the required data. Think of "using index ONLY". See also "*Clustering Data*" on page 111.

PRIMARY (in the "key" or "possible_keys" column)
: `PRIMARY` is the name of the automatically created index for the primary key.

Sorting and Grouping

using filesort (in the "Extra" column)
: "using filesort" in the Extra column indicates an explicit sort operation—no matter where the sort takes place (main memory or on disk). "Using filesort" needs large amounts of memory to materialize the intermediate result (not pipelined). See also "Indexing Order By" on page 130.

Top-N Queries

implicit: no "using filesort" in the "Extra" column
: A MySQL execution plan does not show a top-N query explicitly. If you are using the `limit` syntax and don't see "using filesort" in the extra column, it is executed in a pipelined manner. See also "Querying Top-N Rows" on page 143.

Distinguishing Access and Filter-Predicates

The MySQL database uses three different ways to evaluate `where` clauses (predicates):

Access predicate (via the "key_len" column)
: The access predicates express the start and stop conditions of the leaf node traversal.

Index filter predicate ("Using index condition", since MySQL 5.6)
: Index filter predicates are applied during the leaf node traversal only. They do not contribute to the start and stop conditions and do not narrow the scanned range.

Table level filter predicate ("Using where" in the "Extra" column)
: Predicates on columns which are not part of the index are evaluated on the table level. For that to happen, the database must load the row from the table first.

MySQL execution plans do not show which predicate types are used for each condition—they just list the predicate types in use.

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

In the following example, the entire **where** clause is used as access predicate:

```
CREATE TABLE demo (
   id1 NUMERIC
 , id2 NUMERIC
 , id3 NUMERIC
 , val NUMERIC);

INSERT INTO demo VALUES (1,1,1,1);
INSERT INTO demo VALUES (2,2,2,2);

CREATE INDEX demo_idx
          ON demo
             (id1, id2, id3);

EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id2=1;

+------+----------+---------+------+-------+
| type | key      | key_len | rows | Extra |
+------+----------+---------+------+-------+
| ref  | demo_idx | 12      |    1 |       |
+------+----------+---------+------+-------+
```

There is no "Using where" or "Using index condition" shown in the "Extra" column. The index is, however, used (`type=ref, key=demo_idx`) so you can assume that the entire **where** clause qualifies as access predicate.

You can use the `key_len` value to verify this. It shows that the query uses the first 12 bytes of the index definition. To map this to column names, you "just" need to know how much storage space each column needs (see "Data Type Storage Requirements" in the MySQL documentation). In absence of a `NOT NULL` constraint, MySQL needs an extra byte for each column. After all, each `NUMERIC` column needs 6 bytes in the example. Therefore, the key length of 12 confirms that the first two index columns are used as access predicates.

When filtering with the `ID3` column (instead of the `ID2`) MySQL 5.6 and later use an index filter predicate ("Using index condition"):

```
EXPLAIN
 SELECT *
   FROM demo
  WHERE id1=1
    AND id3=1;

+------+----------+---------+------+-----------------------+
| type | key      | key_len | rows | Extra                 |
+------+----------+---------+------+-----------------------+
| ref  | demo_idx | 6       |    1 | Using index condition |
+------+----------+---------+------+-----------------------+
```

In this case, the key length of six means only one column is used as access predicate.

Previous versions of MySQL used a table level filter predicate for this query—identified by "Using where" in the "Extra" column:

```
+------+----------+---------+------+-------------+
| type | key      | key_len | rows | Extra       |
+------+----------+---------+------+-------------+
| ref  | demo_idx | 6       |    1 | Using where |
+------+----------+---------+------+-------------+
```


Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

Index

Symbols

2PC, *89*
?, :var, @var (see bind parameter)

A

Access Predicate, *44*
access predicates
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*
adaptive cursor sharing (Oracle), *75*
auto parameterization (SQL Server), *39*

B

B-tree (balanced search tree), *4*
between, *44*
bind parameter, *32*
  contraindications
    histograms, *34*
    LIKE filters, *47*
    partitions, *35*
  for execution plan caching, *32*
  type safety, *66*
bind peeking (Oracle), *75*
bitmap index, *50*
Bitmap Index Scan (PostgreSQL), *175*
Brewer's CAP theorem, *89*

C

CAP theorem, *89*
cardinality estimate, *27*
CBO (see optimizer, cost based)
clustered index, *122*
  transform to SQL Server heap table, *127*
clustering factor, *21*, *114*
  automatically optimized, *133*
clustering key, *123*
collation, *24*
commit
  deferrable constraints, *11*
  implicit for **truncate table**, *163*
  two phase, *89*
compiling, *18*
computed columns (SQL Server), *27*
constraint
  deferrable, *11*
  NOT NULL, *56*
cost value, *18*
count(*)
  often as index-only scan, *120*
  Oracle requires NOT NULL constraint, *57*
COUNT STOPKEY, *145*
cursor sharing (Oracle), *39*

D

data transport object (DTO), *105*
DATE
  efficiently working with, *62*
DBMS_XPLAN, *167*
DEALLOCATE, *174*
DEFERRABLE constraint, *11*
DETERMINISTIC (Oracle), *30*
distinct, *97*
distinct()
  in JPA and Hibernate, *97*
DML, *159*
doubly linked list, *2*
dynamic-update (Hibernate), *164*

E

eager fetching, *96*
eventual consistency, *89*
execution plan, *10*, *165*
  cache, *32*, *75*
  creating
    MySQL, *188*
    Oracle, *166*
    PostgreSQL, *172*
    SQL Server, *180*
  operations
    MySQL, *188*
    Oracle, *167*
    PostgreSQL, *174*
    SQL Server, *182*
explain
  MySQL, *188*
  Oracle, *166*
  PostgreSQL, *172*

F

FBI (see index, function-based)
FETCH ALL PROPERTIES (HQL), *105*
fetch first, *144*
filter predicates
  effects (chart), *81*
  recognizing in execution plans
    Oracle, *170*
    PostgreSQL, *177*
    SQL Server, *185*

full table scan, *13*
    `All` (MySQL), *189*
    `Seq Scan` (PostgreSQL), *174*
    `TABLE ACCESS FULL` (Oracle), *168*
    `Table Scan` (SQL Server), *183*
functions, *24*
    deterministic, *29*
        in partial indexes, *52*
    window, *156*

G

`group by`, *139*
    with PostrgesSQL and the Oracle database and an `ASC`/`DESC` index not pipelined, *140*

H

hash join, *101*
`HASH GROUP BY`, *169*
`HASH JOIN` (Oracle), *169*
`HASH Join` (PostgreSQL), *175*
`Hash Match`, *183*
`Hash Match (Aggregate)`, *184*
heap table, *3*, *122*
    creating in SQL Server, *127*
Hibernate
    eager fetching, *96*
    `ILIKE` uses `LOWER`, *98*
    updates all columns, *164*
hint, *19*

I

`IMMUTABLE` (PostgreSQL), *30*
index
    covering, *117*
    fulltext, *48*
    function-based, *24*
        case insensitive, *24*
        to index mathematical calculations, *77*
    join, *50*
    limits
        MySQL, Oracle, PostgreSQL, *121*
        SQL Server, *122*
    merge, *49*
    multi-column, *12*
        wrong order (effects), *81*
    partial, *51*
    prefix (MySQL), *121*
    secondary, *123*
`index` in MySQL execution plans, *189*
index-only scan, *116*
index-organized table, *122*
    database support, *127*
`Index Cond` (PostgreSQL), *177*
`INDEX FAST FULL SCAN`, *168*
`INDEX FULL SCAN`, *168*
`Index Only Scan` (PostgreSQL), *174*
`INDEX RANGE SCAN`, *167*
`Index Scan` (PostgreSQL), *174*
`Index Seek`, *182*
`INDEX UNIQUE SCAN`, *167*
    when accessing an IOT, *124*
`INTERNAL_FUNCTION`, *67*
IOT (index-organized table), *122*

J

join, *91*
    full outer, *109*

K

`Key Lookup (Clustered)`, *182*

L

lazy fetching
    for scalar attributes (columns), *104*
leaf node, *2*
    split, *160*
LIKE, *45*
    alternatives, *48*
    as index filter predicate, *112*
    on DATE column, *67*
    on `DATE` columns, *67*
`limit` (MySQL, PostgreSQL), *144*
logarithmic scalability, *7*
`LOWER`, *24*

M

`Merge Join`, *109*
    PostgreSQL, *175*
    SQL Server, *183*
`MERGE JOIN` (Oracle), *169*
multi-block read
    for a full table scan, *13*
    for a `INDEX FAST FULL SCAN`, *168*
MVCC, *163*
    affects PostgreSQL index-only scan, *174*
myths
    dynamic SQL is slow, *72*, *74*
    most selective column first
        disproof, *43*
        origin, *49*
    Oracle cannot index `NULL`, *56*

N

N+1 problem, *92*
`Nested Loops`, *92*
    PostgreSQL, *175*
    SQL Server, *183*
`NESTED LOOPS` (Oracle), *168*

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

NOSORT
SORT GROUP BY, *140*
WINDOW, *157*
NULL
indexing in Oracle, *54*

O

offset (MySQL, PostgreSQL), *148*
optimizer, *18*
cost based, *18*
hint, *19*
rule based, *18*
statistics, *21*
OPTIMIZE FOR (SQL Server), *76*
OPTION (SQL Server), *76*
OR
to disable filters, *72*
order by, *130*
ASC, DESC, *134*
NULLS FIRST/LAST, *137*
support matrix, *138*
OVER(), *156*

P

paging, *147*
offset method, *148*
seek method, *149*
approximated, *152*
parameter sniffing (SQL Server), *76*
parsing, *18*
partial index, *51*
partial objects (ORM), *104*
partitions and bind parameters, *35*
pipelining, *92*
PLAN_TABLE, *166*
predicate information, *20*
access vs. filter predicates, *44*
in execution plans
MySQL, *190*
Oracle, *170*
SQL Server, *185*
prepare (PostgreSQL), *172*
primary key w/o unique index, *11*

Q

query planner (see optimizer)

R

RBO (see optimizer, rule based)
RECOMPILE (SQL Server hint), *76*
result set transformer, *98*
RID, *3*
RID Lookup (Heap), *183*
root node, *5*
split, *160*
row sequencing, *113*
row values, *153*
ROWID, *3*
ROWNUM (Oracle pseudo column), *144*, *148*
ROW_NUMBER, *156*

S

scalability, *81*
horizontal, *87*
logarithmic, *7*
Scalability, *79*
Seek Predicates (SQL Server), *185*
select *, avoid to
enable index-only scans, *120*
improve hash join performance, *104*
Seq Scan, *174*
Sort (SQL Server), *184*
SORT GROUP BY, *169*
NOSORT, *140*
SORT ORDER BY, *130*
STOPKEY, *145*
SQL area, *75*
SQL injection, *32*
SSD (Solid State Disk), *90*
statistics, *21*
for Oracle function-based indexes, *28*
STATISTICS PROFILE, *181*
STOPKEY
COUNT, *145*
SORT ORDER BY, *146*
WINDOW, *157*
Stream Aggregate, *184*

T

top (SQL Server), *145*
Top-N Query, *143*
TO_CHAR(DATE), *66*
TRUNC(DATE), *62*
truncate table, *163*
triggers not executed, *163*

U

UPPER, *24*

V

Vaccum (PostgreSQL), *163*
virtual columns for NOT NULL constraints on FBI, *58*

W

where, *9*
conditional, *72*
in SQL Server execution plan, *187*
window functions, *156*

SQL Performance Explained! – Now what?

Maybe you still have some questions or a very specific problem that "SQL Performance Explained" did not answer satisfactory? Instant Coaching is the solution for you.

Instant Coaching

Instant Coaching is the fast, easy and hassle-free way for developers to resolve difficult SQL database performance issues via desktop sharing.

Instant Coaching doesn't use made up examples; real cases are presented as this is the optimal way to help you in solving your current problems with slow SQL databases. Instant Coaching is vendor independent and efficient… and offered at a very affordable price!

Give it a try!

Now is the time to learn more about Instant Coaching! Visit our web site for more information:

http://winand.at

Ex Libris GHEORGHE GABRIEL SICHIM <gabriel.sichim@gmail.com>

SQL Performance Explained

SQL Performance Explained helps developers to improve database performance. The focus is on SQL—it covers all major SQL databases without getting lost in the details of any one specific product.

Starting with the basics of indexing and the `where` clause, SQL Performance Explained guides developers through all parts of an SQL statement and explains the pitfalls of object-relational mapping (ORM) tools like Hibernate.

Topics covered include:

- Using multi-column indexes
- Correctly applying SQL functions
- Efficient use of `LIKE` queries
- Optimizing join operations
- Clustering data to improve performance
- Pipelined execution of `order by` and `group by`
- Getting the best performance for pagination queries
- Understanding the scalability of databases

Its systematic structure makes SQL Performance Explained both a textbook and a reference manual that should be on every developer's bookshelf.

Covers

✓ Oracle® Database ✓ SQL Server® ✓ MySQL ✓ PostgreSQL

About Markus Winand

Markus Winand has been developing SQL applications since 1998. His main interests include performance, scalability, reliability, and generally all other technical aspects of software quality. Markus currently works as an independent trainer and coach in Vienna, Austria. http://winand.at/

ISBN 978-3-9503078-2-5

EUR 29.95
GBP 26.99

9 783950 307825 >